

Singleton Pattern in Java Spring Boot Microservices

Singleton Pattern in Java Spring Boot Microservices (0-7 Years Guide)

1. What Is Singleton

Ensures a class has **only one instance** in the application, with a single global access point. Used when creating multiple instances would be wasteful, inconsistent, or incorrect — shared config, connection pools, caches, loggers, registries.

2. Classic Java Implementation (Without Spring)

Basic Thread-Safe Version

```
public class DatabaseConnection {
    private static DatabaseConnection instance;

    private DatabaseConnection() { } // private constructor blocks external instantiation

    public static synchronized DatabaseConnection getInstance() {
        if (instance == null) {
            instance = new DatabaseConnection();
        }
        return instance;
    }
}
```

Bill Pugh Holder Idiom (Preferred — No Locking Overhead)

```
public class DatabaseConnection {
    private DatabaseConnection() { }

    private static class Holder {
        private static final DatabaseConnection INSTANCE = new DatabaseConnection();
    }

    public static DatabaseConnection getInstance() {
        return Holder.INSTANCE;
    }
}
```

The JVM guarantees the inner class loads lazily and only once — thread-safe with no synchronized cost.

Enum Singleton (Most Robust Against Reflection/Serialization Attacks)

```

public enum ConfigManager {
    INSTANCE;

    private final Map<String, String> settings = new HashMap<>();

    public String get(String key) {
        return settings.get(key);
    }
}

```

3. How Spring Boot Handles This For You

Every `@Component`, `@Service`, `@Repository`, `@Controller`, `@RestController`, and `@Bean` is a **singleton by default**, managed by the Spring container — no manual code needed.

```

@Service
public class PaymentService {
    // Spring creates exactly one instance and reuses it everywhere
}

```

Key distinction (often asked in interviews): Spring's singleton is scoped to the **ApplicationContext (the container)**, not the JVM/classloader like the textbook GoF Singleton. One app can have multiple containers, each with its own "singleton" instance.

Bean Scopes

```

@Service
@Scope("singleton") // default, rarely written explicitly
public class ConfigService { }

@Service
@Scope("prototype") // new instance every time it's
// injected/requested
public class RequestHandler { }

```

Other scopes: request, session, application (web-aware, only valid in web contexts).

4. Where Singleton Is Actually Used in Spring Boot Microservices

a) Every Default Spring Bean

```

@Service
public class OrderService { }

@Repository
public class OrderRepository { }

```

Stateless logic classes — safe and efficient to share one instance.

b) Configuration / Properties Beans

```

@Component
@ConfigurationProperties(prefix = "app.payment")
public class PaymentConfig {
    private String apiKey;
    private int timeout;
}

```

Loaded once at startup, read everywhere.

c) Caches

```
@Service
public class ProductCacheService {
    private final Map<Long, Product> cache = new ConcurrentHashMap<>
();
    // must be singleton – a second instance means inconsistent
    cache
}
```

d) Connection Pools / HTTP Clients

```
@Configuration
public class RedisConfig {
    @Bean
    public JedisPool jedisPool() {
        return new JedisPool(new JedisPoolConfig(), "localhost",
6379);
    }
}
```

JedisPool, RestTemplate, WebClient, DataSource — all singleton beans; per-request creation would be catastrophic for performance.

e) Loggers / Metrics Registries

```
@Component
public class MetricsRecorder {
    private final MeterRegistry registry; // Micrometer's
    MeterRegistry is itself a singleton bean
}
```

f) Rate Limiters / Circuit Breaker Registries

```
@Configuration
public class ResilienceConfig {
    @Bean
    public CircuitBreakerRegistry circuitBreakerRegistry() {
        return CircuitBreakerRegistry.ofDefaults(); // tracks state
    }
    across all calls
}
```

Must be singleton — it tracks failure counts over time; a fresh instance per request means the breaker could never trip.

g) Service Registry Clients (Microservices)

```
@Configuration
public class DiscoveryConfig {
    @Bean
    public EurekaClient eurekaClient() {
        // one client maintains the registry connection for the
    whole app
    }
}
```

h) Kafka/RabbitMQ Producers & Consumers

```
@Configuration
public class KafkaProducerConfig {
    @Bean
    public KafkaTemplate<String, String> kafkaTemplate() {
        return new KafkaTemplate<>(producerFactory());
        // one shared template – reusing the underlying connection
    }
    pool
}
```

```
}  
}
```

i) Distributed Lock Managers (Redisson, etc.)

```
@Configuration  
public class RedissonConfig {  
    @Bean  
    public RedissonClient redissonClient() {  
        return Redisson.create(); // single client manages  
distributed locks app-wide  
    }  
}
```

5. Common Mistake — Misusing Singleton for Per-Request State

```
// BAD – mutable instance field on a singleton bean  
@Service  
public class OrderProcessor {  
    private Order currentOrder; // shared across every  
thread/request – race condition!  
  
    public void process(Order order) {  
        this.currentOrder = order; // one user's request can  
overwrite another's  
    }  
}
```

Since the bean is shared across all requests/threads, storing per-request state as an instance field causes data corruption under concurrent traffic.

Fix: pass state as method parameters, or use `@Scope("request")` / `ThreadLocal` for true per-request state.

6. Quick Rule of Thumb

Use Singleton (default) for	Avoid Singleton for
Services, Repositories, Controllers	Anything holding per-request/session mutable state
Config holders, caches, connection pools	Objects meant to be short-lived/discarded
Registries (metrics, circuit breakers, service discovery clients)	Stateful builders reused across threads
Kafka/Redis clients, distributed lock managers	Request-scoped data holders

7. Singleton by Experience Level

0-2 Years (Junior)

Expected to know: - Definition of Singleton and why you'd use it - That Spring beans are singleton by default - Basic `getInstance()` implementation

Common interview questions: - What is the Singleton pattern? - Why are Spring beans singleton by default? - Difference between `@Component`, `@Service`, `@Repository`? - Write a basic thread-safe Singleton in plain Java.

2-4 Years (Mid-Level)

Expected to know: - Is Spring's singleton thread-safe? (No — the *instance* is shared, but thread safety of *state* is the developer's responsibility) - Bean scopes: singleton vs prototype vs request vs session - Practical usage: caches, config beans, HTTP clients as singletons - Identifying when a "singleton" bean is being misused with mutable state

Common interview questions: - Is a Spring singleton bean thread-safe? Explain with an example. - When would you use `@Scope("prototype")` instead of the default? - Show a real case where you used a singleton bean for a cache or client. - What happens if you put mutable instance fields in a `@Service`?

4-7 Years (Senior)

Expected to know: - How to break Singleton (reflection, serialization, cloning) and how to defend against it (Enum singleton, `readResolve()`, overriding `clone()` to throw) - The prototype-bean-injected-into-singleton-bean problem and its fixes (`@Lookup` method injection, `ObjectProvider`, `Provider<T>`) - Distinguishing Spring's container-scoped singleton from the classic GoF/JVM-scoped singleton - Designing shared, thread-safe infrastructure components (circuit breaker registries, distributed lock clients, Kafka templates) correctly across a microservice - Trade-offs of singleton state in a distributed, horizontally-scaled environment (in-memory singleton cache is per-instance, not cluster-wide — need Redis/Hazelcast for shared state across pods)

Common interview questions: - How would you break a Singleton, and how do you prevent each way? - Why can injecting a prototype-scoped bean into a singleton bean cause a memory leak or stale-state bug? How do you fix it? - In a microservice running multiple pods/replicas, is an in-memory singleton cache actually "single"? What's the implication, and how would you fix it? - Design a thread-safe rate limiter/circuit breaker as a singleton bean — walk through the class. - Why does Spring's singleton scope differ from the traditional Singleton design pattern?

8. The Prototype-in-Singleton Problem (Senior-Level Deep Dive)

```
@Component
@Scope("prototype")
public class TaskContext {
    private String taskId;
    // meant to be a fresh instance every time
}

@Service
public class TaskRunner {
    @Autowired
    private TaskContext taskContext; // WRONG — injected only ONCE,
    at singleton creation time
    // every call to run() reuses the same TaskContext instance —
    defeats "prototype"
}
```

Fix 1 — Method Injection with `@Lookup`:

```
@Service
public abstract class TaskRunner {
    @Lookup
    protected abstract TaskContext createTaskContext(); // Spring
    overrides this at runtime
}
```

```

        public void run() {
            TaskContext taskContext = createTaskContext(); // fresh
instance every call
        }
    }
}

```

Fix 2 — ObjectProvider (cleaner, no abstract class needed):

```

@Service
public class TaskRunner {
    private final ObjectProvider<TaskContext> taskContextProvider;

    public TaskRunner(ObjectProvider<TaskContext>
taskContextProvider) {
        this.taskContextProvider = taskContextProvider;
    }

    public void run() {
        TaskContext taskContext = taskContextProvider.getObject();
// fresh instance every call
    }
}

```

9. Microservice-Scale Consideration (Important Senior Point)

An in-memory Singleton bean is **singleton per JVM instance**, not singleton across your whole microservice deployment. If you run 5 replicas/pods of the same service, you have 5 separate “singleton” caches/registries — each isolated. For state that must be truly shared cluster-wide (distributed cache, distributed rate limiter, distributed lock), you need external shared storage:

```

@Configuration
public class RedissonConfig {
    @Bean
    public RedissonClient redissonClient() {
        return Redisson.create(); // singleton client, but backs
onto shared Redis state
    }
}

```

This is a frequent senior-level trap question: *“Your circuit breaker works fine locally but doesn’t behave consistently in production with 5 pods — why?”* → Because each pod has its own in-memory Singleton state; failures on one pod don’t count toward another pod’s breaker.